avoids unnatural formats like `Input:\n <text> Output:\n <text>`. We also allow for adding conditions that force constants (separators, spaces, etc.) in different applications of B_i to be equal. When measuring sensitivity to format perturbations, if two separators or spaces are equal in an original format, they are forced to jointly change to be considered equivalent. Appendix A.3 contains all contextual restrictions.

Final Prompt Construction. Given a valid format p accepted by the grammar, the final prompt is constructed by concatenating with space c an instruction string $inst$, n few-shot data points $D_1, \ldots, D_n$ exemplifying the task, and a data point D_{n+1} to be solved. All few-shot examples D_i are formatted using p. Thus, the final prompt template is: $inst\ c\ p(D_1)\ c\ p(D_2)\ c\ \ldots\ c\ p(D_n)\ c\ p(D_{n+1})$. Since D_{n+1}'s output will be generated by the model, an empty string is added in place of the answer in the last field in the template. Prompt construction will modify $inst$ to match specific choices encoded in p: concretely, if p enumerates valid multiple-choice options as characters $x_1 \ldots x_n$, we ensure $inst$ refers to these choices as $x_1 \ldots x_n$.

3.2 Measuring Sensitivity

We measure how plausible choices in prompt formatting influence quantifiable metrics of generated outputs. Given a set of plausible formats $\{p_1, \ldots, p_n\}$, a dataset $\mathcal{D}$, and a scalar metric m, let the *performance interval* be $[\min_i m(p_i, \mathcal{D}), \max_i m(p_i, \mathcal{D})]$. We define the *performance spread* or simply *spread* as $\max_i m(p_i, \mathcal{D}) - \min_i m(p_i, \mathcal{D})$. Higher spread indicates more sensitivity to variance within the space of plausible, semantically-equivalent formats. While our method is agnostic to the scalar metric m used, and one could consider a number of metrics including text length, formality, or toxicity, throughout this work we focus our analysis on estimated task accuracy acc. Due to ease in automatic evaluation, here we evaluate on classification tasks.

Our goal is to compute spread for a given model and task. A comprehensive approach would be to fully evaluate each plausible format p_i on the entire evaluation dataset $\mathcal{D}$. This increases the cost of reporting a model's performance linearly with n, which becomes computationally infeasible for large values of n. Following prior gradient-free prompt engineering work (Zhou et al., 2023; Pryzant et al., 2023), we model our problem as a multi-arm bandit. Given a random sample of n formats (arms) $p_1, \ldots, p_n$ for a task, an arm p_i's hidden value is the actual performance $m(p_i, \mathcal{D})$ when evaluated on the full dataset $\mathcal{D}$, and the reward for pulling the arm is an estimate $m(p_i, \tilde{\mathcal{D}})$ where $\tilde{\mathcal{D}} \subset \mathcal{D}$, $|\tilde{\mathcal{D}}| = B$ (mini-batch size) and no element of $\tilde{\mathcal{D}}$ has yet been evaluated with p_i.

We assume a budget of E total data point evaluations. We first search for the highest performing format with budget $E/2$, and then for the lowest performing format with budget $E/2$. Evaluations done for the first exploration are readily available for the second exploration, which yields a more informative prior for many formats. We consider two well-known regret minimization bandit algorithms: Thompson sampling (used in FormatSpread) and Upper Confidence Bound (UCB).

Thompson Sampling. This simple, high-performing Bayesian inference heuristic randomly draws each arm according to its probability of being optimal (Chapelle & Li, 2011). Each $m(p_i, \mathcal{D})$ is modeled as a random variable, and since with our target metric each data point evaluation is a Bernoulli trial, it is natural to model $m(p_i, \mathcal{D})$ as a Beta distribution. In each round, Thompson sampling draws from each $m(p_i, \tilde{\mathcal{D}})$ and chooses the best arm $\hat{i}$ (Algorithm 1). It then updates $\hat{i}$ according to the number of observed successes r, and the corresponding $B - r$ failures, within $\tilde{\mathcal{D}}$.

Algorithm 1 Thompson Sampling for Bernoulli Bandits

$S_i^{(1)} \leftarrow 0, N_i^{(1)} \leftarrow 0$ (success counters and total times armed was drawn counter)
for $t \leftarrow 1, \ldots E/B$ **do**
  for $i \leftarrow 1, \ldots, K$ **do**
    Take $\theta_i^{(t)}$ from Beta$(\alpha_i + S_i^{(t)}, \beta_i + (N_i^{(t)} - S_i^{(t)}))$
  Draw arm $\hat{i} = \arg\max_i \theta_i^{(t)}$ (or arg min in minimization problems) and observe reward r
  $S_{\hat{i}}^{(t+1)} \leftarrow S_{\hat{i}}^{(t)} + r, \quad N_{\hat{i}}^{(t+1)} \leftarrow N_{\hat{i}}^{(t)} + B$

Thompson sampling allows for setting informative priors (α_i, β_i) based on domain knowledge to accelerate runtime. Appendix A.4 details the exact priors we use. To our knowledge, we are the first to consider a Bayesian sampling method for prompt optimization.

Upper Confidence Bound (UCB) Sampling. UCB (Lai et al., 1985) computes an upper confidence bound to each arm's performance, derived from Chernoff's bound. The key difference with Thompson sampling is in how $\theta_i^{(t)}$ is defined. In UCB's frequentist approach, $\theta_i^{(t)}$ is assigned the estimated accuracy plus the upper confidence bound: $\theta_i^{(t)} \leftarrow S_i/N_i + c\sqrt{log(t)/N_i}$. We use $c = 2$ following Pryzant et al. (2023), who find UCB with $c = 2$ to be most effective for prompt optimization.

Naive Sampling. Each prompt format is evaluated on E/n points (with appropriate rounding).

4 CHARACTERIZING PROMPT FORMAT VARIANCE WITH FORMATSPREAD

4.1 EXPERIMENTAL SETUP

Data. We use a subset of 53 tasks from Super-NaturalInstructions (Wang et al., 2022) with diverse human-written formats and instructions, comprising 19 multiple-choice tasks and 34 classification tasks with $\{2, 3, 4\}$ basic fields. Appendix B.1 details the exact task selection procedure. To construct the final prompt template, we concatenate each task's instruction and n formatted few-shot examples using `\n\n` as spacing. While selection and ordering of few-shot examples is a component of prompt design influencing features of model output (Lu et al., 2022), our work focuses on prompt formatting. To remove this confounder, we fix the exact choice and ordering of examples for each task and for a given number of shots n. Few-shot examples for each task are chosen randomly within each dataset and are not used for evaluation. We evaluate task data samples on an arbitrary order fixed across settings. Datasets are assumed to be of size 1,000 for fair evaluation across tasks.

Models. We evaluate LLaMA-2-{7B,13B,70B} (Touvron et al., 2023), Falcon-7B and Falcon-7B-Instruct (Almazrouei et al., 2023), GPT-3.5-Turbo (Schulman et al., 2022), all autoregressive LMs.

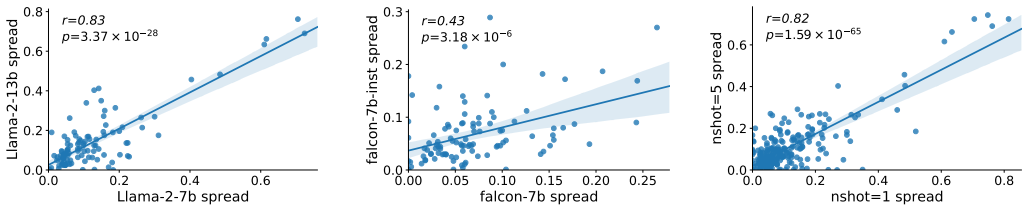
Task Evaluation Metrics. We use two popular measures for computing accuracy: exact prefix matching and probability ranking. In exact prefix matching, we check if the output's prefix matches the expected answer after normalization (casing, spacing, newlines). Ranking accuracy computes the rate that the expected answer is the highest-ranked valid option (in multiple choice and classification tasks) according to the model's output distribution. Results are reported using ranking accuracy unless specified otherwise. Appendix B.2 shows additional analysis of exact prefix matching, with spreads even higher than those shown in Section 4.2, and including how formatting choice affects task degeneration (i.e., not answering any valid option).

4.2 PROMPT FORMATS HAVE A LARGE PERFORMANCE SPREAD, NOT ELIMINATED BY INCREASING FEW-SHOT EXAMPLES OR MODEL SIZE, NOR WITH INSTRUCTION TUNING

For each evaluation task we randomly sample 10 plausible prompt formats and use FORMATSPREAD to compute performance spread for each modeling and n-shot choice (Figure 3). We find significant performance spread across tasks, with a median spread of 7.5 accuracy points across choices in the model and the number of few-shot examples. 20% of tasks consistently result in a spread of at least 15 accuracy points for all LLaMA-2 settings, and at least 9 points for all Falcon settings. We observe several tasks with performance spread over 70 accuracy points. Because this analysis uses only 10 randomly sampled formats, it represents a lower bound of the true spreads for each task. Furthermore, there exists significant performance spread regardless of increased model size (Figure 2a and Figure 11 for Llama-2-70B), instruction tuning (Figure 2b), or number of few-shot examples (Figure 2c; Figure 2a and 2b plot 1- and 5-shot jointly). Appendix B.2 demonstrates similar results on a selection of non-classification tasks, and expands the spread discussion to plotting the entire accuracy distribution, along with a dispersion metric.

Comparison trends between models are often reversed just by choosing different formats. Assuming model M is better than M' by at least d accuracy using prompt p, we compute how often M' achieves at least d higher accuracy than M under a different format p'. Figure 4 shows these trends are often reversed: LLaMA-2-13B and -70B reverse trend by at least $d = 0.02$ with probability 0.141; LLaMA-2-7B and Falcon-7B reverse trend by at least $d = 0.02$ with probability 0.140. Strikingly, often both experiments (first using p, and then p') were statistically significant (p-value < 0.05) on 1000 samples[2]: 76% and 47% respectively for the two model comparisons

[2]We use one-sided McNemar tests, also known as paired χ^2 tests, since we evaluate models on the same set of samples. We test the significance of M being *better* than M' under p, and M being *worse* than M' under p'.



(a) Llama-2-7B vs. 13B (b) Falcon-7B vs. 7B-Instruct (c) 1- vs. 5-shot (same task, model)

Figure 2: Spread comparison between evaluating the same task under different models or n-shots.

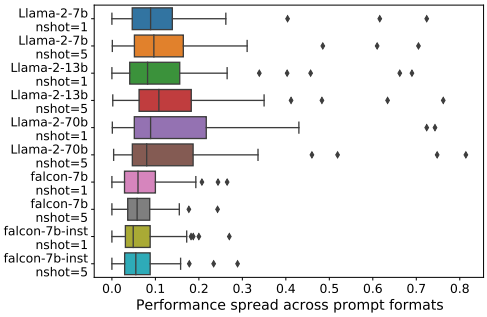


Figure 3: Spread across models and n-shots.

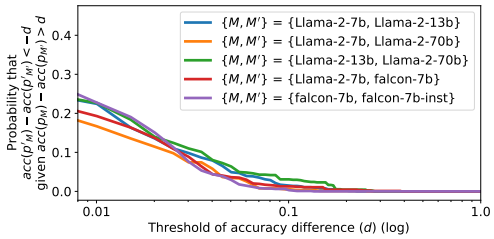


Figure 4: Probability that model M performs *worse* than M' by at least d when using format p', given that M performed *better* than M' by at least d using format p. 53 tasks, 1- and 5-shot.

mentioned above. We find that formats yielding high performance for model M may not yield high performance for M', implying that **formats may not be inherently good or bad** (Appendix B.2).

4.3 How do individual features contribute to performance?

We analyze how choices in particular constants (i.e. $\mathcal{S}_1, \mathcal{S}_2, \mathcal{C}, \mathcal{F}_{\text{casing}}, \mathcal{F}_{\text{item}}$) independently influence task performance across different formats. Figure 5 shows the distribution of accuracy for 500 sampled prompts conditioned on the choice of $\mathcal{S}_1$ (the separator between a descriptor and the text placeholder) for one task in Super-NaturalInstructions. When comparing the individual influence of two feature choices, we measure both *weak* and *strong* notions of dissimilarity between distributions of accuracy across prompts conditioned on a chosen feature. We say two constant choices yield *weakly* different accuracy distributions if the values between the first quartile (Q_1) and third quartile (Q_3) do not intersect. This is equivalent to the boxes in a boxplot not overlapping. We say two constant choices yield *strongly* different accuracy distributions if the ranges $[2.5Q_1 - 1.5Q_3, 2.5Q_3 + 1.5Q_1]$ do not overlap (adjusted to end in a data point). This is equivalent to two boxplots with their whiskers not overlapping. In Figure 5, `' \n\t'` and `': '` (fourth and sixth) are only weakly different.

We compute accuracy for 500 random formats with 250 samples each on 31 tasks for 1-shot Llama-2-7B. Table 1 shows that choices in $\mathcal{S}_2$, $\mathcal{F}_{\text{item1}}$, $\mathcal{F}_{\text{casing}}$ do not independently predict performance differences (weakly or strongly): although these features can have a large performance variance and thus should be explored with FormatSpread, they cannot be used to independently predict accuracy changes. Other constant sets have varying degrees of differences, with $\mathcal{S}_1$ (separators) and $\mathcal{F}_{\text{item2}}$ (number format changes in enumerations) having the most individual impact. All tasks with strong dissimilarities are shown in Appendix B.4.

Small prompt variations often yield large performance differences. Table 2 shows a selection of tasks where changing a single constant on a format (e.g., casing in task322) results in large accuracy differences. Figure 6 shows that regardless of the scoring criterion used, a significant ratio of these atomic changes are associated with large accuracy changes. For example, 24% of atomic changes have an associated accuracy change of at least 5 points when using exact prefix matching as scoring criteria (11% when using probability ranking).

The space of prompt format accuracy is highly non-monotonic, which makes local search algorithms over the space less effective. Let (p_1, p_2, p_3) be a prompt format triple such that p_{i+1} is obtained by making an atomic change to p_i. We argue that if the prompt format space is smooth, we should often see a triples' accuracy to be strictly monotonic over i. We choose 24 tasks (13 multiple choice,